

To prevent this recursion, the code inside your getter, setter, and deleter methods should always act on the backing variable (which should have an underscore prefix in its name), never the property. Code outside these methods should use the property, although as with the private access underscore prefix convention, nothing prevents you from writing code on the backing variable anyway.

Using Setters to Validate Data

The most common need for using properties is to *validate* data or to make sure it's in the format you want it to be in. You might not want code outside the class to be able to set an attribute to just any value; this could lead to bugs. You can use properties to add checks that ensure only valid values are assigned to an attribute. These checks let you catch bugs earlier in code development, because they raise an exception as soon as an invalid value is set.

Let's update the *wizcoin.py* file from Chapter 15 to turn the `galleons`, `sickles`, and `knuts` attributes into properties. We'll change the setter for these properties so only positive integers are valid. Our `WizCoin` objects represent an amount of coins, and you can't have half a coin or an amount of coins less than zero. If code outside the class tries to set the `galleons`, `sickles`, or `knuts` properties to an invalid value, we'll raise a `WizCoinException` exception.

Open the *wizcoin.py* file that you saved in Chapter 15 and modify it to look like the following:

```
❶ class WizCoinException(Exception):
❷     """The wizcoin module raises this when the module is misused."""
    pass

class WizCoin:
    def __init__(self, galleons, sickles, knuts):
        """Create a new WizCoin object with galleons, sickles, and knuts."""
❸     self.galleons = galleons
        self.sickles = sickles
        self.knuts = knuts
        # NOTE: __init__() methods NEVER have a return statement.

--snip--
```